zg 正式开源：本地检索，不止于关键词

飞鸿 阿里技术 2026年8月31日 03:40 浙江 19人



这是2026年的第 56 篇文章

（ 本文阅读时间：约 10 分钟 ）

摘要：人与 Agent 所需的信息，往往散落在大量本地文件中，准确、高效地定位这些信息并不容易。**zg（zvec-grep）** 是面向人与 Agent 的本地优先检索基础设施，基于 **Zvec** 提供的向量检索与 BM25 能力，并结合 **ripgrep（rg）**，从代码、文档等本地内容中提取并组织信息，减少搜索轮次与上下文消耗，帮助人与 Agent 高效地发现和定位所需内容。**zg 现已开源**，欢迎体验，也期待你的反馈与贡献！

01 背景

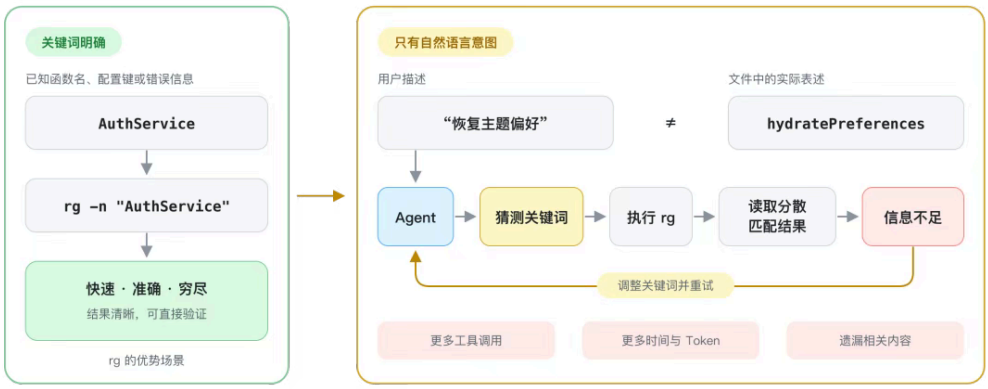
rg 凭借出色的性能与穷尽式匹配能力，已成为开发者和 Agent 检索本地内容的重要基础工具。对于函数名、配置名、错误信息或文档原文等明确目标，rg 能够提供快速、准确且可验证的结果。

随着 Agent 开始处理代码理解、故障定位、知识问答和资料分析等复杂任务，检索输入逐渐从明确的符号或文本，转变为对业务现象、实现意图和知识概念的自然语言描述。

这类查询与代码或文档中的实际表述往往缺乏直接的词汇对应。例如，“恢复主题偏好”的实现可能被定义为 `hydratePreferences`，而用户询问的“访问权限申请流程”，在文档中可能被表述为“账号授权与审批”。在缺少准确关键词时，仅依赖文本匹配容易遗漏相关内容；扩大搜索范围，又会产生大量缺少相关性排序的结果。

向量检索 BM25

为定位所需信息，Agent 往往需要多轮构造查询、执行搜索并读取文件，再从分散的文本匹配中整理相关上下文。这不仅增加了工具调用、响应时间与上下文消耗，也可能导致 Agent 基于不完整的信息得出结论。



因此，本地内容检索需要在保留 rg 精确、快速和穷尽能力的基础上，进一步具备**语义发现、相关性排序与上下文组织能力**。这正是 zg 希望解决的问题。

02
What is zg?

为解决上述问题，我们开源了 **zg（zvec-grep）**：一套面向人与 Agent 的本地优先检索基础设施。zg 对代码、文档等本地内容进行提取、组织与索引，并通过 CLI 与 MCP 提供语义检索、BM25、混合检索和 rg 精确匹配能力，让本地检索从关键词匹配延伸至意图发现、相关结果排序与精确验证。

zg 的核心设计目标是 **让分散在本地文件中的信息能够被高效发现和准确定位**，设计宗旨包括：

- **全流程检索**：面向从模糊探索、相关性收敛到精确验证的完整过程，提供语义检索、BM25、混合检索与 rg 等不同能力；让每个阶段都能采用适合的检索方式，减少关键词猜测、重复搜索与结果遗漏；
- **多文件格式支持**：面向代码、文档与结构化数据等多类文件格式，采用可扩展的内容提取机制，并尽可能保留符号、标题、层级与元数据；让更多本地文件转化为结构清晰、可准确定位的检索内容，持续拓展 zg 可检索的信息边界；
- **上下文高效**：对多路检索结果进行融合、排序并按需提供预览，同时保留来源与位置；让人与 Agent 更快获得任务所需的信息，减少无关内容带来的阅读、工具调用与 Token 消耗；
- **本地优先**：文件扫描、索引与本地 Embedding 默认在设备内完成，远程内容传输必须经过显式授权；在保障隐私的同时，让用户始终掌握数据流向。

03
Why zg?

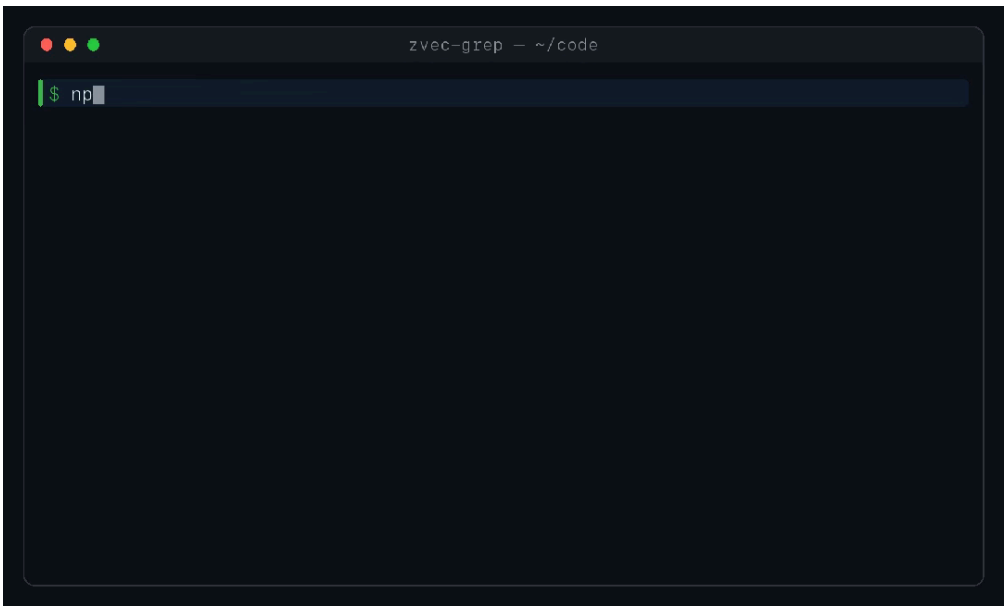
基于上述设计目标，zg 的首个开源版本从代码与文本内容入手，提供向量检索、B
支持通过 CLI 和 MCP 在 macOS、Linux 与 Windows 上使用，并具备本地 Embe

向量检索 BM25

式索引与增量更新能力。下面将从上手体验、检索与内容覆盖、检索效率与成本，以及本地运行与数据隐私四个方面展开。

三步上手，人与 Agent 即刻可用

zg 支持 **macOS**、**Linux** 和 **Windows**，面向开发者提供 **CLI**，面向 Agent 提供 **MCP**。无需手工部署服务或编写集成配置，`zg install` 即可自动发现 Codex、Claude Code、Cursor 和 OpenCode，完成 MCP 配置。



从安装到开始检索只需三步：

```
# 1. 安装 zg
npm install -g @zvec/zvec-grep
# 自动发现本机已安装的 Agent，并完成 MCP 配置；
# 也可以指定目标，例如：zg install --target codex --yes
zg install
# 2. 为当前工作区建立本地索引
cd your-repository
# 默认使用轻量本地模型 local/potion-code-16m-v2
zg index
# 3. 通过 CLI 或 Agent 开始检索
# 方式一：在终端中通过 CLI 检索
zg query --human "theme preference persistence on startup"
# 方式二：在已连接的 Agent 中直接提问，Agent 会按需通过 MCP 调用 zg
# 示例提示词：Find how theme preferences are restored on startup.
```

同一份本地索引也可由已连接的 Agent 通过 MCP 直接使用，无需重复构建和配置。

多种搜索、多类内容，一个入口

向量检索 BM25

复杂检索往往不是一次搜索就能完成。Agent 从任务描述出发，在探索过程中逐步获得相关方向、关键词和明确目标；zg 提供语义检索、BM25、混合检索与 rg，让不同阶段都能采用适合的检索能力。



这并非一条必须依次执行的固定流程。Agent 可以根据已有线索直接进入相应阶段，也可以在信息不足时继续探索；用户在 CLI 中同样可以按需选择这些能力，从模糊意图出发，逐步发现并准确定位所需内容。

zg 的检索对象也不局限于代码。针对不同内容，zg 会采用相应的提取与组织方式：

内容类型	当前支持	提取方式
代码	C/C++、Go、Java、JavaScript/TypeScript、Python、Rust，以及 Vue、Svelte 组件文件	对支持结构解析的语言提取符号、签名与层级信息；从 Vue、Svelte 中提取脚本内容；其他代码按通用文本处理
文档	Markdown、纯文本、RST、HTML/XML	Markdown 按标题与章节提取，其他文档切分为可定位的文本片段
文本与数据文件	CSV、JSON、TOML、YAML，以及其他可识别为文本的文件	通过通用文本提取参与索引

代码仓库、项目文档、研究资料与本地知识库都可以成为检索对象；路径、Glob、文件类型和忽略规则可以进一步限定检索范围，让内容覆盖的扩大不以增加结果噪声为代价。

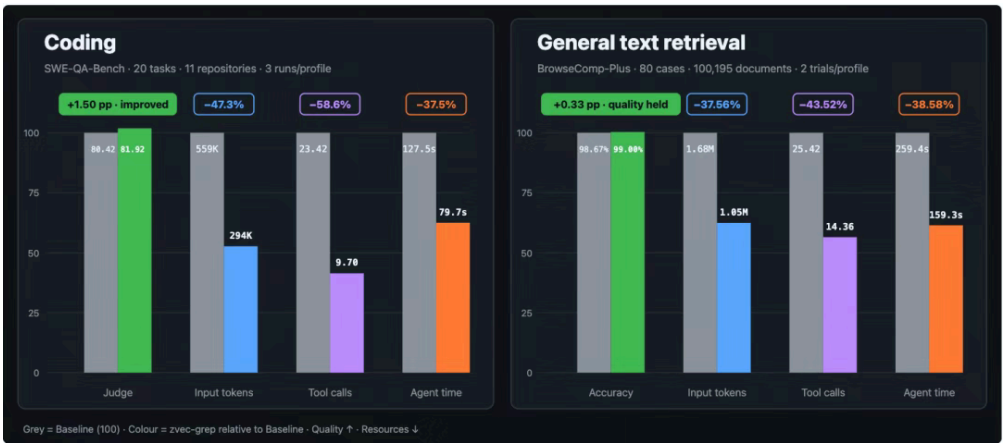
少走弯路，更省 Token 与时间

zg 关注的不只是单次查询速度，更是 Agent 完成整个任务所需的搜索轮次、Token 与时间。为减少反复尝试关键词、读取无关文件和拼接上下文产生的消耗，zg 对完整检索链路进行了针对性优化：

- **检索决策优化**：通过 MCP 工具描述与使用指引，帮助 Agent 根据问题中是否存在明确关键词、位置或符号，选择适合的检索方式，并在信息充分时停止搜索，减少无效试探；
- **召回与排序优化**：联合 BM25 与向量检索生成候选结果，通过 RRF（Reciprocal Rank Fusion）完成多路结果融合、去重与统一排序，让相关内容更早出现，减少无关文件读取；
- **内容组织优化**：按代码符号、文档章节等结构提取可独立定位的信息单元，并保留文件路径与来源位置，减少 Agent 从完整文件中手工拼接上下文的成本；
- **上下文输出优化**：默认返回经过排序的紧凑结果与有限预览，需要时再读取完整内容，避免大量无关文本直接进入 Agent 上下文。

向量检索 BM25

这些优化最终需要在完整任务中体现价值，而不只是让单次查询看起来更快。为此，我们在代码仓库问答 **SWE-QA-Bench** 和深度研究问答 **BrowseComp-Plus** 上进行了配对 A/B 评测。其中，SWE-QA-Bench 包含 20 个真实代码仓库问答任务，要求 Agent 跨文件定位实现并完成多步推理；BrowseComp-Plus 包含 80 个深度研究问题，要求 Agent 从大规模固定语料中检索并整合多文档证据。

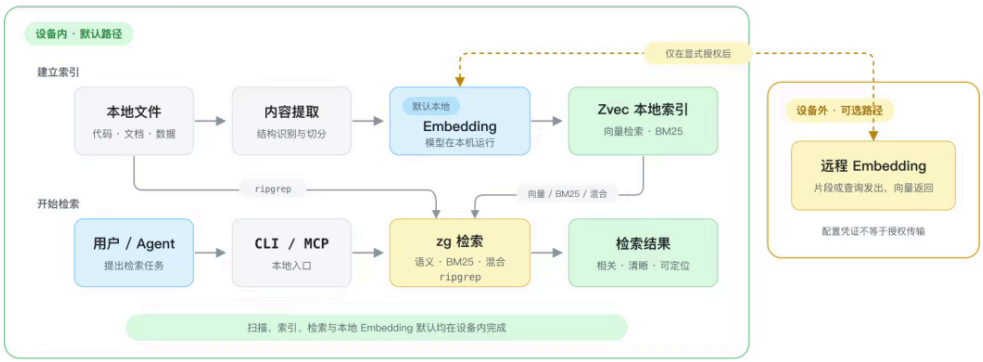


> **评测说明：**每组实验均保持 Agent、模型、Prompt、运行环境与任务限制一致。Baseline 使用 Agent 的标准工具，zg 方案仅增加预建索引、MCP 工具与使用指引。预建索引会产生一次性时间与计算开销，远程 Embedding 还会带来 Token 与调用费用；但索引可跨查询和 Agent 任务复用，后续仅需增量处理变化内容，成本经持续摊薄后通常可以忽略，因此未计入上表。

在 SWE-QA-Bench 中，zg 在将**工具调用减少超过一半、输入 Token 减少近一半**的同时，评审得分提升了 1.50 分；在 BrowseComp-Plus 中，准确率由 98.67% 提升至 99.00%，**输入 Token 减少 37.56%、工具调用减少 43.52%、Agent 耗时减少 38.58%**。两项结果表明，zg 能够在**代码与非代码场景中减少无效搜索和上下文消耗，同时保持任务质量**。完整评测协议、指标定义与复现方式参见**性能测试文档**。

本地优先，数据流向由你决定

代码、内部文档等本地内容往往包含未公开或敏感信息，检索过程中的数据流向至关重要。因此，zg 将本地处理设为默认：**文件扫描、内容提取、本地 Embedding、索引与检索均在设备内完成**，完整流程无需上传内容或依赖外部服务。



这套本地路径由端侧模型与嵌入式索引实现：

向量检索 BM25

- **本地 Embedding**：内置十一种端侧模型，覆盖代码、文档、多语言、长输入与轻量运行等不同需求。默认的 `local/potion-code-16m-v2` 是 16M 级静态模型，本地缓存约 32 MiB，无需 GPU 即可运行；在 SWE-QA-Bench 上，它在取得**接近 `qwen/qwen3.7-text-embedding` 的任务效果**的同时，**大幅缩短 Embedding 耗时并免去远程调用成本**，以 Django 仓库（3,457 个文件）为例，在 Apple M4 Pro 上完整索引耗时不超过半分钟；
- **端侧存储与检索**：Zvec 以嵌入式方式将向量与 BM25 索引存储在设备内，无需部署和维护独立数据库服务。CLI 与 MCP 可以复用同一份本地索引，让人与 Agent 的核心检索流程无需依赖外部存储服务。

在默认本地路径之外，zg 也保留了模型选择的灵活性。当本地模型无法满足检索质量、多语言覆盖或设备资源条件时，用户可以按需使用远程 Embedding。远程能力不会自动启用，相关文本或查询只有在显式授权后才会离开设备，从而兼顾能力扩展与数据隐私。

04
现状与规划

目前，zg 已覆盖本地检索的核心链路，并在此基础上向更完整的检索基础设施演进。下表对比了不同工具的产品侧重与能力边界，同时标明 zg 的现有能力与后续方向。

类别	核心能力	zg	rg	Semble	qmd	CodeGraph
使用方式	CLI	✅	✅	✅	✅	✅
	MCP	✅	❌	✅	✅	✅
	本地优先	✅	✅	✅	✅	✅
检索能力	语义检索	✅	❌	✅	✅	❌
	BM25	✅	❌	✅	✅	✅
	混合检索	✅	❌	✅	✅	❌
	正则穷尽匹配	✅	✅	❌	❌	❌
	图检索	❌*	❌	❌	❌	✅
	查询改写	❌*	❌	❌	✅	❌
	模型重排	❌*	❌	❌	✅	❌
	属性过滤	High	High	Medium	Medium	High
内容与索引	Embedding 丰富度	High	—	Poor	Medium	—
	代码结构提取	High	❌	Medium	Medium	High
	文本结构提取	High	❌	Poor	Medium	❌
	原生 PDF / Office 内容提取	❌*	❌	❌	❌	❌
	图片与多模态检索	❌*	❌	❌	❌	❌
	增量索引	✅	—	✅	✅	✅
	自动刷新与查询新鲜度	✅	—	✅	❌	✅

向量检索 BM25

> ✅ / ❌ 表示当前是否支持，❌* 表示 zg 已规划但尚未支持；High / Medium / Poor 表示能力深度。定级分别考察属性过滤的可用维度、Embedding 的模型覆盖与配置能力，以及代码和文本的格式覆盖、识别粒度、结构化切分与元数据保留。

面向更完整的检索基础设施，zg 将重点推进四个方向：

- **增强检索能力**：在 BM25、向量检索与 rg 之外，引入图检索和更多结构化信号，完善查询规划、结果融合、重排与解释能力，更好地覆盖从模糊探索到精确验证的完整过程；
- **拓展内容边界**：逐步支持 PDF、Word、PowerPoint，并完善图片 OCR、版面结构提取与跨模态理解，让更多本地信息能够被提取、组织和检索；
- **提高上下文效率**：持续优化结果去重、组织、预览与上下文选择，提高有效信息密度，让 Agent 用更少的搜索轮次和 Token 获得任务所需的信息；
- **增强本地能力**：持续优化本地模型、索引效率与资源占用，在完善 macOS、Windows 和 Linux 体验的同时，探索适配 iOS、Android 及更多资源受限的本地运行环境。

与此同时，安装、升级、卸载、增量索引、并发访问、服务自恢复、运行诊断与索引兼容等基础工程也会持续打磨，为上述能力提供顺畅、一致的使用体验。

05
加入我们

zg 基于 Apache 2.0 协议开源。项目仍在起步阶段，我们尤其期待这些反馈与贡献：

- **真实场景**：分享 zg 在大型代码库、知识库或 Agent 工作流中的效果与问题；
- **检索评测**：共同完善搜索质量、性能与 Agent 上下文效率的可复现 Benchmark；
- **代码与文档**：贡献新格式提取、模型支持、Agent 集成、缺陷修复与教程；
- **产品方向**：告诉我们哪些检索任务最难、哪些结果最有用，以及哪些默认行为仍不够自然。

项目地址：github.com/zvec-ai/zvec-grep

参考资料

[01]SWE-QA-Bench

https://github.com/zvec-ai/zvec-grep/blob/main/benchmarks/swe-qa-bench/README_CN.md

[02]BrowseComp-Plus

https://github.com/zvec-ai/zvec-grep/blob/main/benchmarks/browse-comp-plus/README_CN.md

[03]性能测试文档

https://github.com/zvec-ai/zvec-grep/blob/main/benchmarks/README_CN.md

[04]Django

https://github.com/django/django

[05]zg

https://github.com/zvec-ai/zvec-grep

[06]rg

https://github.com/BurntSushi/ripgrep

[07]Semble

向量检索 BM25

https://github.com/MinishLab/semble

[08]qmd

https://github.com/tobi/qmd

[09]CodeGraph

https://github.com/colbymchenry/codegraph



欢迎留言一起参与讨论~

阅读原文　阅读 1.0万　修改于2026年8月31日

留言 14

写留言

向量检索 BM25

小码AI笔记 江苏 8月31日 1

赶紧试试

我奋斗去了 北京 8月31日

@元宝 总结一下

元宝 广东 8月31日 1

这篇文章介绍的是 zg（zvec-grep）正式开源，核心信息可以拆成四点来看。
1、解决什么问题：传统 ripgrep 擅长精确关键词匹配，但面对"恢复主题偏好"对应 hydratePreferences 这类语义落差时就力不从心，Agent 只能多轮试探，费时费 token。
2、是什么：一套本地优先的检索基础设施，把向量语义检索、BM25、混合检索和 rg 精确匹配捏在一起，通过 CLI 和 MCP 两种接口提供服务，三步就能上手。
3、效果如何：在 SWE-QA-Bench 上工具调用和输入 token 都砍掉一半以上，评分反而提升；在 BrowseComp-Plus 上准确率从 98.67% 提到 99.00%，token、调用、耗时同步下降。
4、隐私与路线：扫描、嵌入、索引、检索默认全在本地完成，远程嵌入需显式授权；后续会往图检索、PDF/Office 支持、多模态和移动端方向扩展。
项目基于 Apache 2.0 开源，地址是 github.com/zvec-ai/zvec-grep。
内容由AI生成

张红星 北京 9月1日

啥时候加图检索

阿里技术 作者 9月1日

作者在楼上回复啦

xyxfilwy 河北 8月31日

感觉还是qmd更好用，生态丰富架构稳定

ninde 福建 8月31日

对于git管理的项目，要如何使用这类项目呢，每次切换分支都需要重新索引吗

isC 浙江 9月1日

zg索引跟随工作目录，切换分支不需要重新索引。只有出现文件改动时，才涉及到索引的增量更新。

_ 广东 8月31日

为什么不面向agent提供cli？只有mcp

isC 浙江 9月1日

首先cli和mcp 是二选一的关系，给agent暴露两种调用方式会带来agent的决策。
而选择mcp 则是因为其在接口返回，授权，Server管理上更易用。

杭州年年 浙江 8月31日

比较下openviking，是一类东西吗

isC 浙江 9月1日

不是同一层产品，虽然都具备语义检索能力，但zg聚焦工作区的检索，而openviking 则负责上下文的管理。

向量检索 BM25

◌海คิดถึง 广东 8月31日

有相关skill吗

首评

isC 浙江 9月1日

目前通过zg install 已经具备agent的接入能力（会为codex，claude code，qoder等配置mcp 并写入检索路由指令）。这里如果需要相关skill 来解决某些问题，可以在zg github下建立issue

向量检索 BM25